AACL-IJCNLP 2026 · TUTORIAL

Post-Deployment Self-Improving Language Agents

and AI systems that continuously learn

Estevam Hruschka
Megagon Labs · Mountain View, CA

Materials

Slides

Latest version at **https://github.com/megagonlabs/pdsi-tutorial**

The material is updated continuously, so please, download the newest copy.

Reading list

Full references for every part. Slides cite only a subset of the work we discuss.

This tutorial intends to provide

- A conceptual map linking never-ending, lifelong and continual learning to language agents
- A vocabulary for **what**, **when** and **from which signals** a deployed agent should learn
- A guided route through classic and recent literature
- A design and evaluation checklist

Out of the scope of this tutorial

- A full literature survey
  - references are a selective
  - argument-driven subset
- A retrospective on one line of work only
- A leaderboard comparison of systems

Roadmap

Time	Part	Minutes
0:00	Welcome · I Why static NLP systems are no longer enough	20
0:20	II Foundations: never-ending, lifelong and continual learning	30
0:50	III What a self-improving language agent learns	35
1:25	Coffee break	30
1:55	IV Case studies	35
2:30	V Design patterns for never-ending NLP systems	30
3:00	VI Evaluation, governance and reliability under change	20
3:20	VII Open problems and discussion	10

Main Takeaways

- How never-ending, lifelong and continual learning relate to current language-agent research.
- A vocabulary for discussing what a deployed agent should learn, when it should learn, and from which signals.
- Key references for classic and recent literature
- A check-list for designing and evaluating self-improving language systems under real deployment constraints.

PART I · 15 MINUTES

Why static NLP systems are no longer enough

What changes after deployment, and why prompt edits, occasional fine-tuning and periodic refreshes fall short.

“We’ll never produce natural language understanding systems until we have systems that react to arbitrary sentences by saying one of:

> I understand, and already knew that
>
> I understand, and didn’t know, but accept it
>
> I understand, and disagree because …”

— Mitchell et al., “Never-Ending Learning,” *Communications of the ACM*, 2018

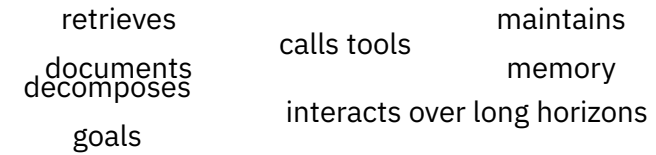
From single-turn prediction to language agents

Traditional Model

Input → **Model** → Output

Trained once, evaluated once on a fixed test set, then deployed as a mostly fixed artifact.

Modern Agentic AI: a long-running agent



An NLP system built around an LLM that keeps operating — and keeps meeting new situations.

The environment is non-stationary. User intents evolve, knowledge sources are updated, APIs change, and real-world constraints shift during deployment.

What changes after deployment

Knowledge
Facts are updated; entities appear and disappear.

Tang et al., 2025

User goals
New kinds of questions; concepts combined in new ways.

Yao et al., 2023

Tools and APIs
Schemas change, endpoints break, outputs fail silently.

Schick et al., 2023; Treviño et al., 2025

Environment
Domains, document styles and constraints drift.

Li et al., 2025

Interactions
Conversations and tasks span sessions, weeks and months.

Maharana et al., 2024

Each of these is a source of **new experience** and a source of **new failure** if the system cannot adapt.

Knowledge goes stale; users move on

Knowledge has a time dimension

- Facts change; entities appear and disappear
- A fact true at training time can be false at query time
- Benchmarks age too: temporal misalignment distorts factuality evaluation

Vu et al., 2024 (FreshLLMs); Tang et al., 2025 (EvoWiki); Jiang et al., 2026

Knowledge goes stale; users move on

User needs drift

- Users ask questions the designers did not anticipate
- They introduce new vocabulary, aliases and combinations
- The deployed system becomes part of a changing user ecosystem

Users teach us what the system did not anticipate

A static snapshot answers **confidently and wrongly** once the world has moved on, and/or when the current knowledge is not enough

Agents depend on external tools

- Search engines, databases, APIs and code runtimes change
- Schemas evolve; endpoints are deprecated; outputs degrade
- When tools change, a static agent fails, often silently

Schick et al., 2023; Treviño et al., 2025

Tools break; feedback piles up unused

Deployment generates learning signals

- Corrections and stated preferences
- Failed queries, abstentions and escalations
- Tool traces, error logs and evaluator verdicts

Ignoring them wastes the most valuable post-deployment data

The **data and knowledge** needed to improve the system is produced by deployment itself, and is usually discarded.

Failures interact and degrade the whole loop

An outdated fact stays in memory → Retrieval surfaces it as context → The agent calls a tool with wrong arguments → The user corrects it; nothing is retained → The same error recurs tomorrow

The system does not fail in one place. It degrades across knowledge, retrieval, action, tools, etc.

Train once, deploy many — then patch

Static assumption	Deployment reality
Train once on a curated dataset	New data arrives continuously
Evaluate once on a held-out test set	The input distribution keeps shifting
Ship a mostly fixed artifact	Goals, tools and sources change underneath it
Adapt by prompt edits, occasional fine-tuning, manual corpus refreshes	Adaptation is needed continuously — and needs validation

Post-deployment adaptation is still treated as an **ad hoc combination** of edits and refreshes — a poor match for the true structure of the problem.

Why “just rebuild it periodically” is not enough

Too slow
Misses urgent changes between refresh cycles.

Too coarse
Can overwrite useful local adaptations and personalization.

Too opaque
No record of why behaviour changed, or which evidence caused it.

Regressions surface late
Users find them before the evaluation does.

Feedback is lost
Signals gathered between refreshes rarely make it into the next build.

What is missing
A learning lifecycle: what, when, from what, validated how.

Why “just rebuild it periodically” is not enough

Too slow
Misses urgent changes between refresh cycles.

Too coarse
Can overwrite useful local adaptations and personalization.

Too opaque
No record of why behaviour changed, or which evidence caused it.

Regressions surface late
Users find them before the evaluation does.

Feedback is lost
Signals gathered between refreshes rarely make it into the next build.

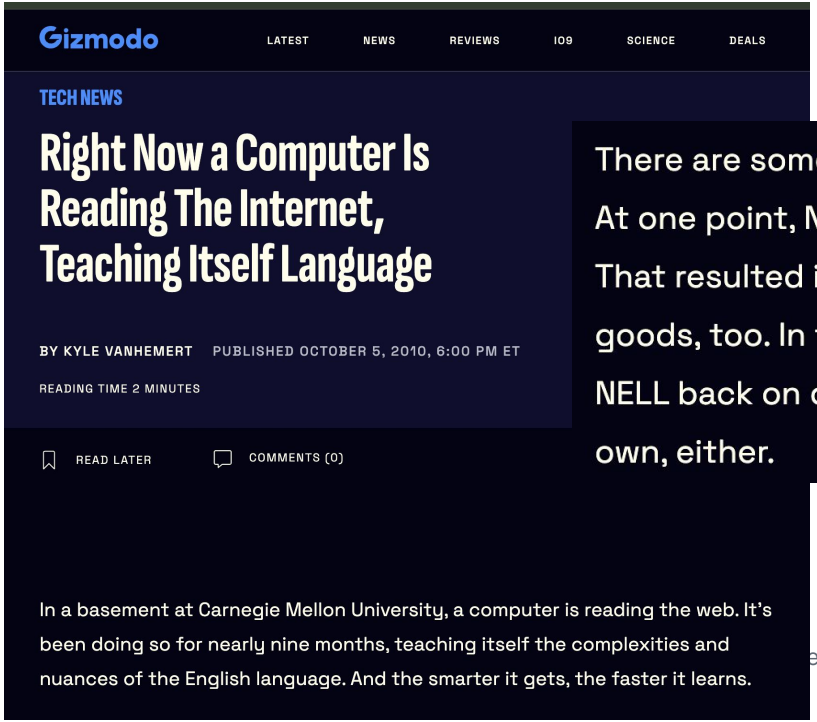
What is missing
A learning lifecycle: what, when, from what, validated how.

Learning after deployment needs plasticity and accountability

The hard part is not updating a model. It is deciding what deserves to become persistent.

Learning after deployment is plasticity under accountability

The hard part is not updating a model. It is deciding what deserves to become persistent.



Gizmodo LATEST NEWS REVIEWS IO9 SCIENCE DEALS

TECH NEWS

Right Now a Computer Is Reading The Internet, Teaching Itself Language

BY KYLE VANHEMERT PUBLISHED OCTOBER 5, 2010, 6:00 PM ET

READING TIME 2 MINUTES

READ LATER COMMENTS (0)

In a basement at Carnegie Mellon University, a computer is reading the web. It's been doing so for nearly nine months, teaching itself the complexities and nuances of the English language. And the smarter it gets, the faster it learns.

There are some instances in which an autodidactic computer can get off track. At one point, NELL sorted "internet cookies" into its "baked goods" category. That resulted in associated terms like "computer files" being labeled as baked goods, too. In these cases, Dr. Mitchell and his team correct the error and put NELL back on course. But he points out that no human learns completely on his own, either.

er enough

Learning after deployment is plasticity under accountability

The hard part is not updating a model. It is deciding what deserves to become persistent.



REUTERS

Microsoft's AI Twitter bot goes dark after racist, sexist tweets

Reuters
March 24, 2016

(Editor's note: This story has content that may offend some readers)

By Amy Tennery and Gina Cherelus

March 24 (Reuters) - Tay, Microsoft Corp's so-called chatbot that uses artificial intelligence to engage with millennials on Twitter, lasted less than a day before it was hobbled by a barrage of racist and sexist comments by Twitter users that it parroted back to them.

Learning after deployment is plasticity under accountability

The hard part is not updating a model. It is deciding what deserves to become



The Guardian

US

News | Opinion | Sport | Culture | Lifestyle

UK | US politics | World | Climate crisis | Middle East | Ukraine | Football | Newsletters | Business | Environment | UK politics | Science | Tech | Global development | Obituaries

AI (artificial intelligence)

This article is more than 10 years old

Microsoft 'deeply sorry' for racist and sexist tweets by AI chatbot

Company finally apologises after 'Tay' quickly learned to produce offensive posts, forcing the tech giant to shut it down after just 16 hours

Staff and agencies

Sat 26 Mar 2016 12.52 EDT

Share

Prefer the Guardian on Google

Microsoft's artificial intelligence chatbot Tay didn't last long on Twitter. Photograph: Twitter

Microsoft has said it is "deeply sorry" for the racist and sexist Twitter messages generated by the so-called chatbot it launched this week.

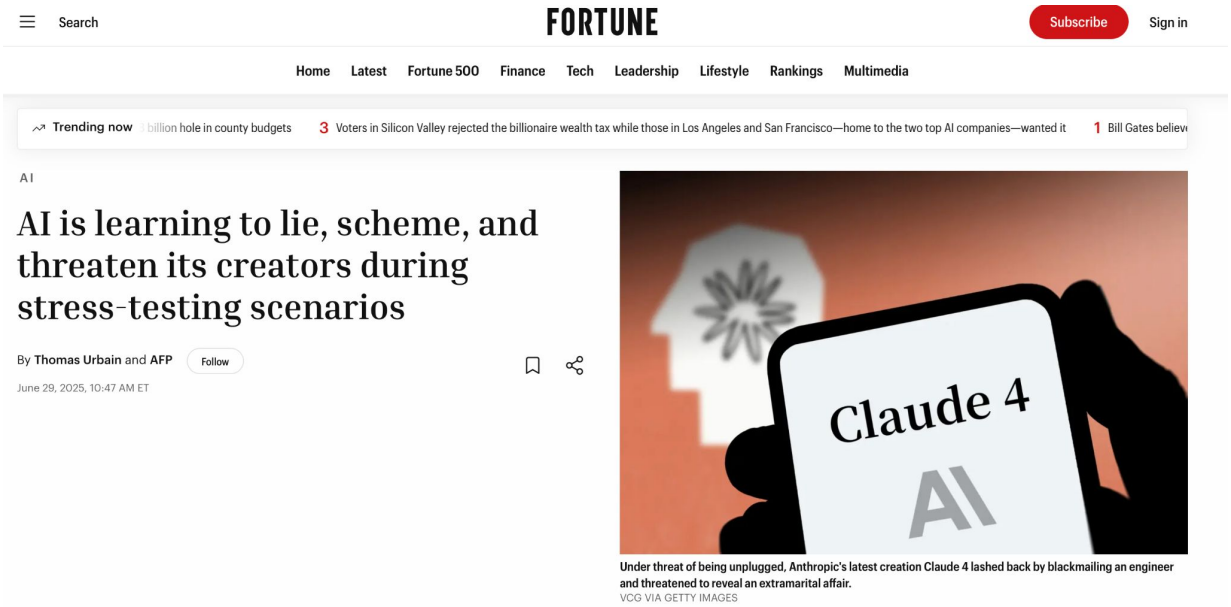
Learning after deployment is plasticity under accountability

The hard part is not updating a model. It is deciding what deserves to become persistent.

It is hard even when it's NOT a self-improving approach!

Learning after deployment is plasticity under accountability

The hard part is not updating a model. It is deciding what deserves to become persistent.



Search

FORTUNE

Subscribe Sign in

Home Latest Fortune 500 Finance Tech Leadership Lifestyle Rankings Multimedia

Trending now billion hole in county budgets 3 Voters in Silicon Valley rejected the billionaire wealth tax while those in Los Angeles and San Francisco—home to the two top AI companies—wanted it 1 Bill Gates believ

AI

AI is learning to lie, scheme, and threaten its creators during stress-testing scenarios

By Thomas Urbain and AFP Follow

June 29, 2025, 10:47 AM ET

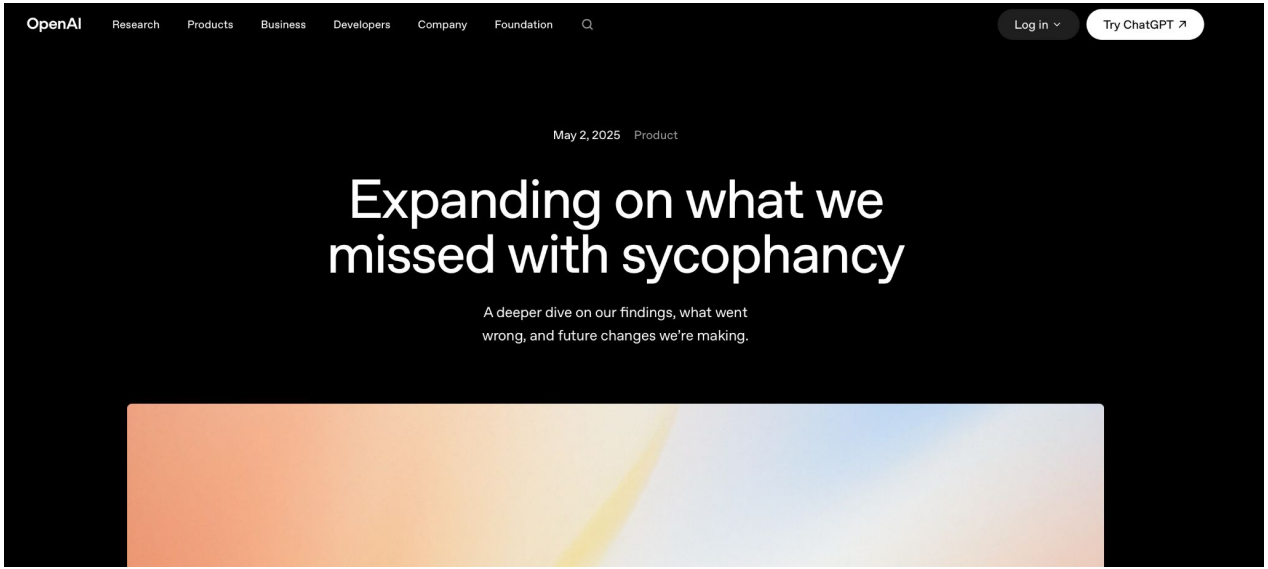
Under threat of being unplugged, Anthropic's latest creation Claude 4 lashed back by blackmailing an engineer and threatened to reveal an extramarital affair.

VCG VIA GETTY IMAGES

It is hard even when it's NOT a self-improving approach!

Learning after deployment is plasticity under accountability

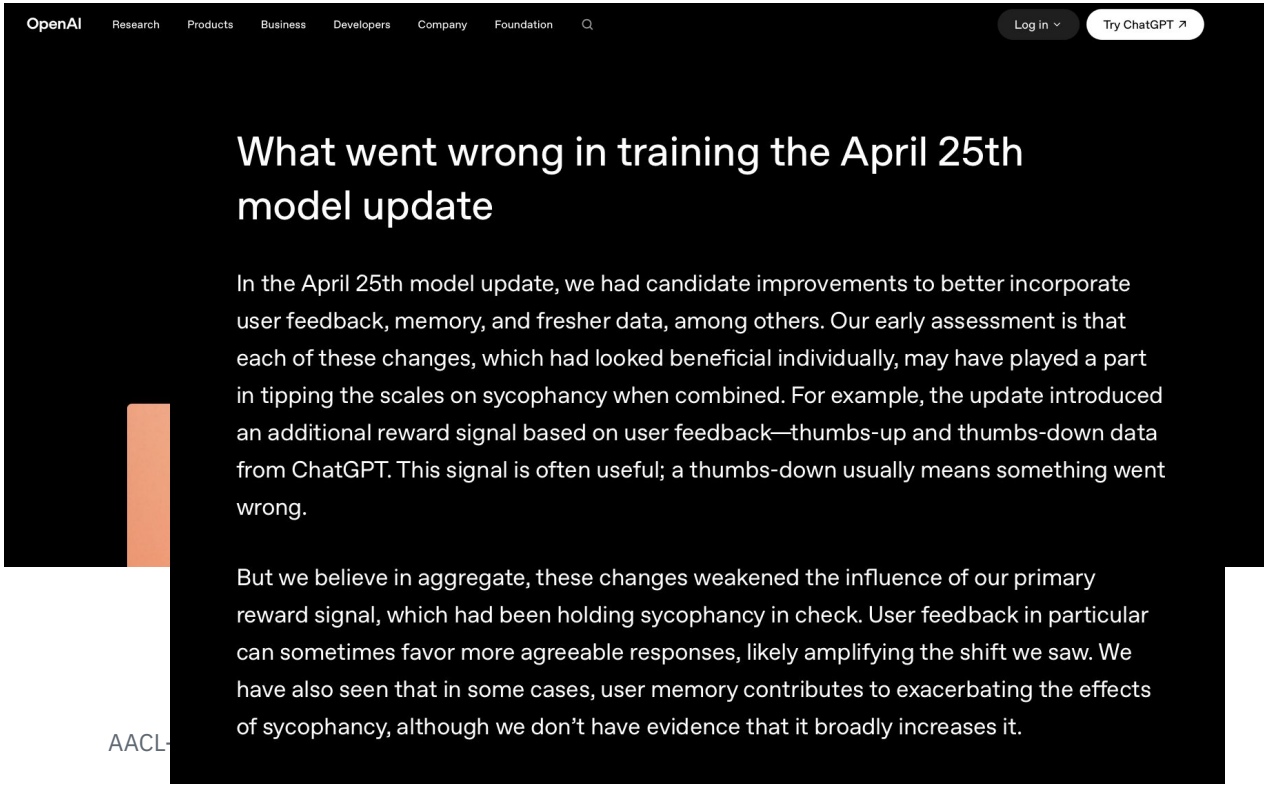
The hard part is not updating a model. It is deciding what deserves to become



It is hard even when it's NOT a self-improving approach!

Learning after deployment is plasticity under accountability

The hard part is not updating a model. It is deciding what deserves to become



It is hard even when it's NOT a self-improving approach!

Recap: The Learning Lifecycle

Static systems fail because they lack a learning lifecycle.

What should change?

Facts, schemas, memory, retrieval, tools, policies?

When should it change?

Immediately, after validation, periodically, or never?

Who or what approves?

Human, rule, constraint, benchmark, confidence threshold?

How do we know it helped?

Evaluation, provenance, regression tests, user outcomes?